Selection Sort

Selection sort is a sorting algorithm that selects the smallest element from an unsorted list in each iteration and places that element at the beginning of the unsorted list.

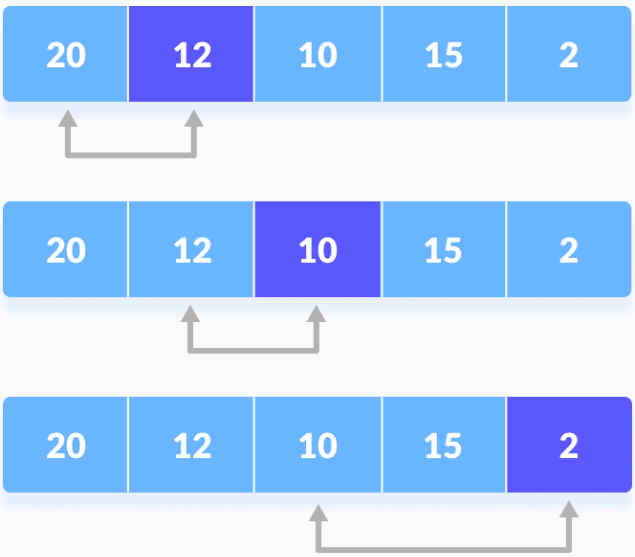
Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the **smallest (or largest)** element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.

1. First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
2. Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
3. We keep doing this until we get all elements moved to correct position.

Working of Selection Sort

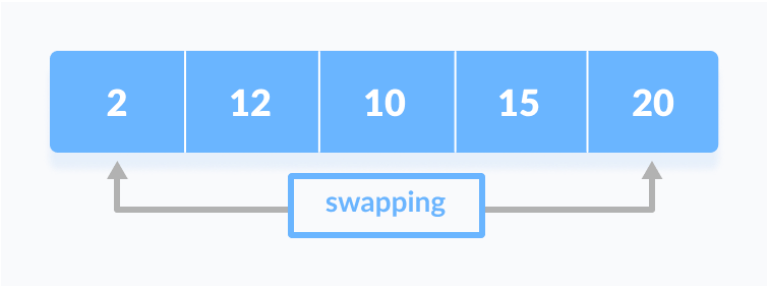
1. Set the first element as minimum.

2. Compare minimum with the second element. If the second element is smaller than minimum, assign the second element as minimum.
   Compare minimum with the third element. Again, if the third element is smaller, then assign minimum to the third element otherwise do nothing. The process goes on until the last element.



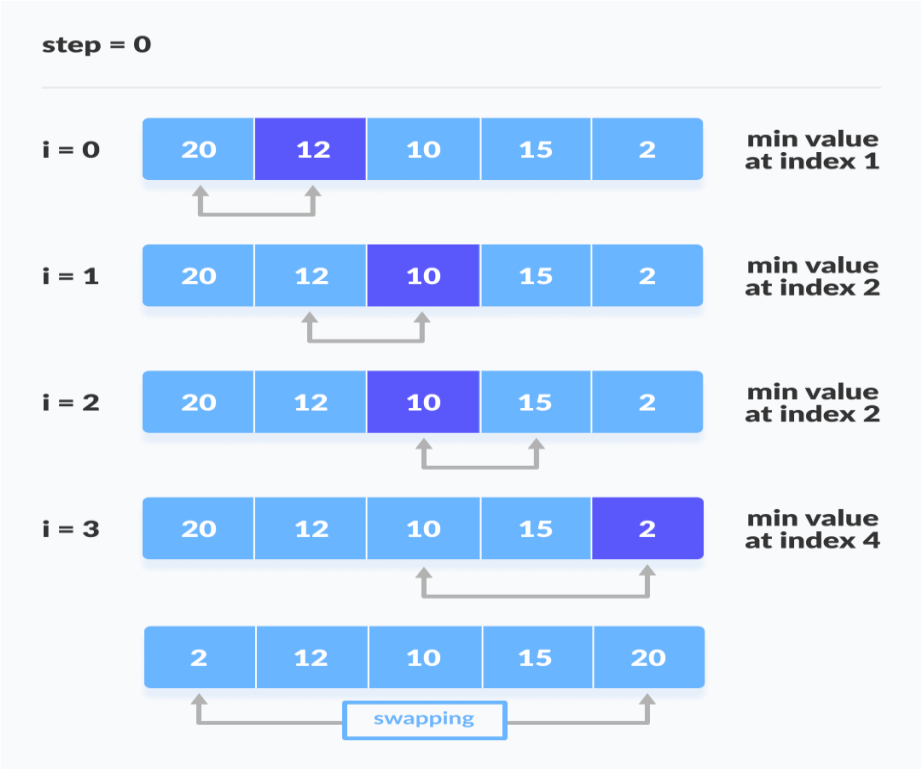
Compare minimum with the remaining elements

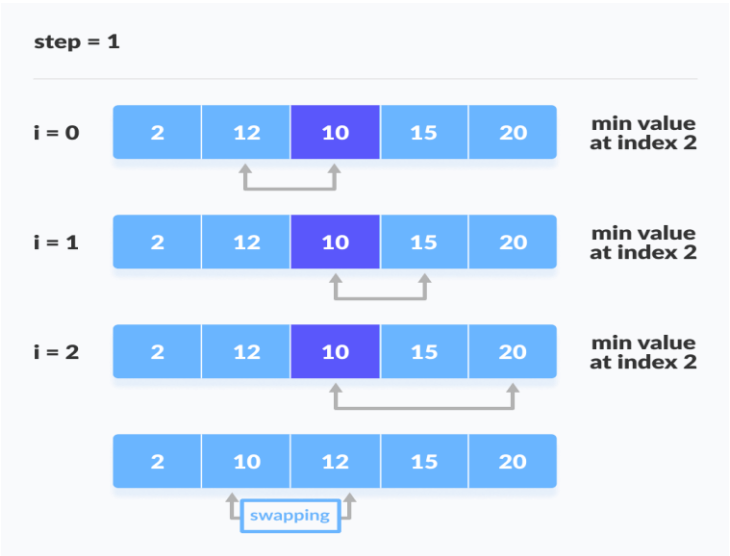
3. After each iteration, minimum is placed in the front of the unsorted list.

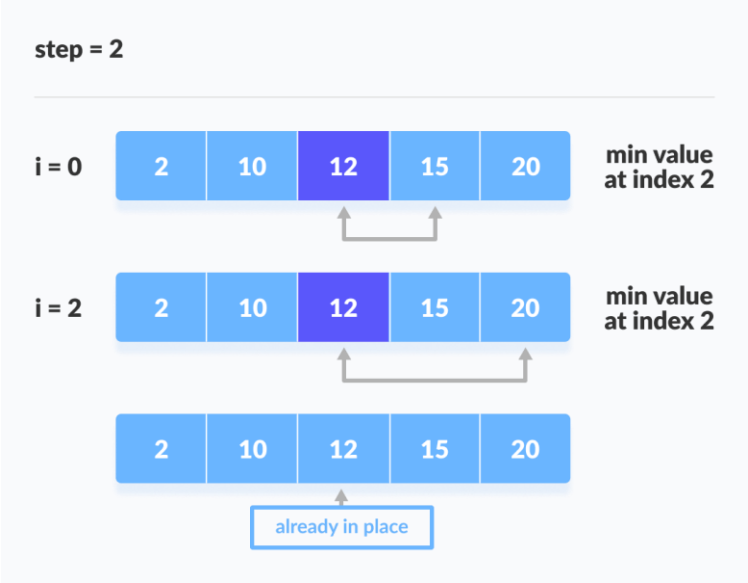


Swap the first with minimum

4. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.







Selection Sort Complexity

Time Complexity	
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

Cycle	**Number of Comparison**
1st	(n-1)
2nd	(n-2)
3rd	(n-3)
...	...
last	1

Number of comparisons: $(n - 1) + (n - 2) + (n - 3) + ... + 1 = n(n - 1) / 2$ nearly equals to n^2.

Complexity = $O(n^2)$

Also, we can analyze the complexity by simply observing the number of loops. There are 2 loops so the complexity is $n*n = n^2$.

Time Complexities:

- **Worst Case Complexity: O(n^2)**
  If we want to sort in ascending order and the array is in descending order then, the worst case occurs.
- **Best Case Complexity: O(n^2)**
  It occurs when the array is already sorted
- **Average Case Complexity: O(n^2)**
  It occurs when the elements of the array are in jumbled order (neither ascending nor descending).
  The time complexity of the selection sort is the same in all cases. At every step, you have to find the minimum element and put it in the right place. The minimum element is not known until the end of the array is not reached.

Space Complexity:

Space complexity is O(1) because an extra variable `temp` is used.

Selection Sort Applications

The selection sort is used when

- a small list is to be sorted
- cost of swapping does not matter
- checking of all the elements is compulsory
- cost of writing to a memory matters like in flash memory (number of writes/swaps is O(n) as compared to O(n^2) of bubble sort)

Python Function to sort data using Selection Sort

```
def selectionSort(a):
  n=len(a)
  for i in range(0,n):
    min = i
    for j in range(i + 1, n):
      if a[j] < a[min]:
        min = j
    (a[i], a[min]) = (a[min], a[i])

a = [64, 34, 25, 5, 22, 11, 90, 12]
selectionSort(a)
print('The array after sorting in Ascending Order by selection sort is:')
print(a)
```

Output:

```
The array after sorting in Ascending Order by selection sort is:
[5, 11, 12, 22, 25, 34, 64, 90]
```